

A
Project Report
On
**RetailPulse – AI-Powered Customer
Analytics &
Demand Forecasting Platform**

**See retail trends, forecast demand,
and
turn every sale into smarter strategy**

Prepared by
Savsani Mann A.

TABLE OF CONTENTS

ABSTRACT.....	1
CHAPTER 1: PROJECT OVERVIEW	2
1.1 VISION AND OBJECTIVES.....	2
1.2 TARGET USERS.....	2
1.3 BUSINESS VALUE DELIVERED.....	2
1.4 NON-FUNCTIONAL FUNCTIONS	2
CHAPTER 2: KEY FEATURES.....	3
CHAPTER 3: TECHNOLOGY STACK.....	4
CHAPTER 4: ARCHITECTURE OVERVIEW	5
CHAPTER 5: DETAILED EXECUTION TIMELINE.....	7
5.1 WEEK-BY-WEEK BREAKDOWN	6
5.2 MAJOR DELIVERABLES PER PHASE	6
CHAPTER 6: TECHNICAL HIGHLIGHTS	9
6.1 MODEL PERFORMANCE METRICS	9
6.2 FEATURE ENGINEERING DETAILS.....	9
6.3 MLOPS PRACTICES.....	9
6.4 CHALLENGES FACED AND SOLUTIONS	9
CHAPTER 7: DEPLOYMENT AND OPERATIONS.....	10
7.1 MLOPS PRACTICES.....	10
7.2 CI/CD PIPELINE SUMMARY	10
7.3 MONITORING DETECTION SETUP	10
CHAPTER 8: VISUALS	11
CHAPTER 9: PERSONAL REFLECTIONS.....	18

LIST OF TABLES

TABLE 2.1 KEY FEATURES	3
TABLE 3.1 TECHNOLOGY STACK	4

ABSTRACT

RetailPulse is an AI-powered analytics platform designed to convert static retail data into an interactive business intelligence experience for stores and operations teams. The product aims to improve decision-making and responsiveness by offering real-time insights to merchandisers, analysts, and managers. Key features include sales trend analysis, customer segmentation, demand forecasting, churn prediction, and inventory optimization, with the latter two using machine learning models for predictive accuracy, while others deliver descriptive analytics and rule-based alerts. Built with Flask and Python for backend processing, Streamlit or Dash for visualization, scikit-learn and XGBoost for modeling, SQLite or Supabase-style storage for the database, and time-series forecasting plus clustering for data science components, RetailPulse ensures a scalable and reliable solution. Users access tailored dashboards to review performance, refine strategy, and improve operational efficiency across the retail lifecycle effectively.

CHAPTER 1: PROJECT OVERVIEW

1.1: Vision and Objectives:

Our vision is to make RetailPulse the intelligent analytics hub that retailers rely on to turn sales, customer, and inventory data into actionable insights. The objectives are to improve decision speed, uncover customer segments, forecast demand, predict churn risk, optimize stock planning, enable data-driven merchandising, and deliver streamlined dashboards and alerts that empower teams to respond faster to market changes and uplift performance across retail operations.

1.2: Target Users:

RetailPulse serves retail managers, merchandisers, inventory planners, analysts, and operations teams who need fast, accurate insights from sales, customer, and stock data. These users rely on the platform to monitor performance, identify trends, allocate inventory, and coordinate promotions. By providing predictive forecasts, customer segmentation, and real-time dashboards, RetailPulse helps teams make data-driven decisions, reduce stockouts, manage churn, and improve overall retail efficiency, responsiveness, and outcomes.

1.3: Business value delivered:

RetailPulse delivers business value by turning fragmented retail data into clear operational advantage, reducing stockouts, and improving inventory turnover through demand forecasting. It boosts revenue by identifying high-value customer segments, supports quicker merchandising decisions, reduces churn risk with predictive analytics, and lowers costs with smarter stock planning. By enabling cross-functional teams to act on real-time insights, RetailPulse increases responsiveness, profitability, and retail execution effectiveness significantly.

1.4: Non-Functional Goals:

RetailPulse aims for high accuracy in demand forecasts and churn predictions, maintaining error margins below industry benchmarks. It targets low latency for dashboard updates and query responses, ideally under two seconds, while supporting concurrent use by dozens of retail teams. Scalability is designed for growing SKU counts and store locations with elastic data handling. The platform also emphasizes reliability, secure access control, and operational uptime for continuous retail decision support.

CHAPTER 2: KEY FEATURES

ID	Feature	Description	Acceptance Criteria
1	Sales Analytics	Visualize historical sales trends and KPIs across stores, products, and time periods.	Users can view sales charts, filter by date/store/product, and see revenue/trend summaries.
2	Customer Segmentation	Group customers using RFM and behavior metrics to identify high-value and churn-risk segments.	Group customers using RFM and behavior metrics to identify high-value and churn-risk segments.
3	Demand Forecasting	Group customers using RFM and behavior metrics to identify high-value and churn-risk segments.	Predict future product demand using time-series models and historical sales data.
4	Churn Prediction	Identify customers likely to churn based on purchase behavior and engagement indicators.	Risk scores are produced for customer records and highlighted in a dedicated churn dashboard.
5	Inventory Optimization	Recommend stock levels and replenishment priorities to reduce stockouts and overstock.	Optimization suggestions appear per SKU/store and include expected impact on stockouts or holding cost.
6	Real-Time Dashboard	Provide up-to-date operational dashboards and alerts for retail teams.	Dashboards refresh quickly, display key metrics, and trigger alerts for threshold breaches.

Table 2.1 Comparative Analysis

CHAPTER 3: TECHNOLOGY STACK

Category	Technology	Rationale	Alternatives
Frontend	Streamlit	Fast Python-based dashboard development for interactive analytics	Dash or Voila
Backend	Python	Core language for data engineering, modeling, and app logic	R or JavaScript/Node.js
Data Science	pandas, NumPy, scikit-learn, Prophet, XGBoost, statsmodels, imbalanced-learn	Provides ETL, feature engineering, forecasting, classification, and imbalance handling	TensorFlow, PyTorch, or AutoML tools
Visualization	Plotly, Matplotlib, Seaborn	Enables interactive charts and static visualizations for dashboards	Bokeh or Altair
MLOps	MLflow, Evidently	Tracks experiments, models, and data drift	Weights & Biases, Neptune, or Seldon
Testing	pytest	Standard Python testing framework for unit and integration tests	unittest or nose

Table 3.1 Technology Stack

CHAPTER 4: ARCHITECTURE OVERVIEW

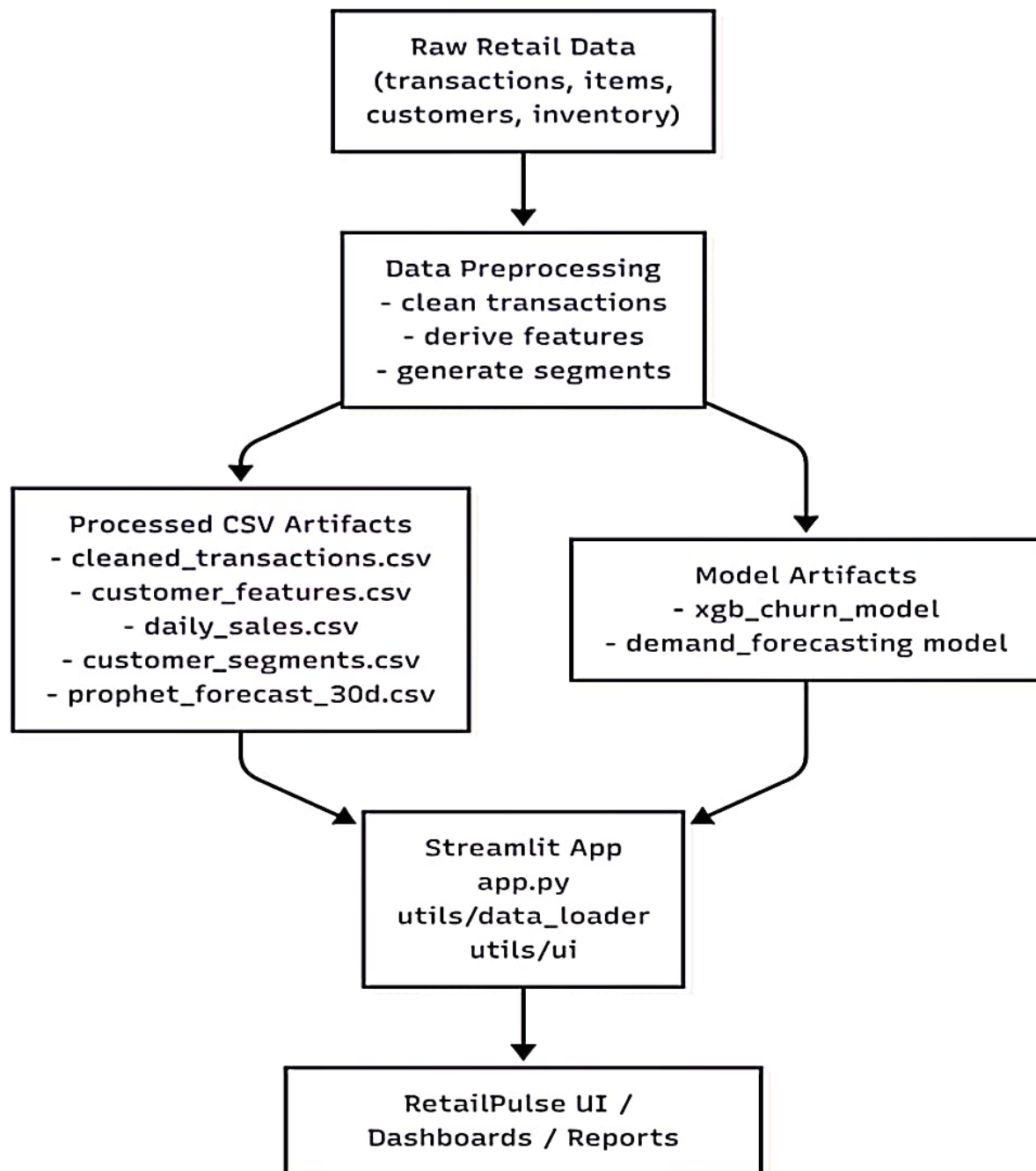


Fig 4.1 Architecture Diagram

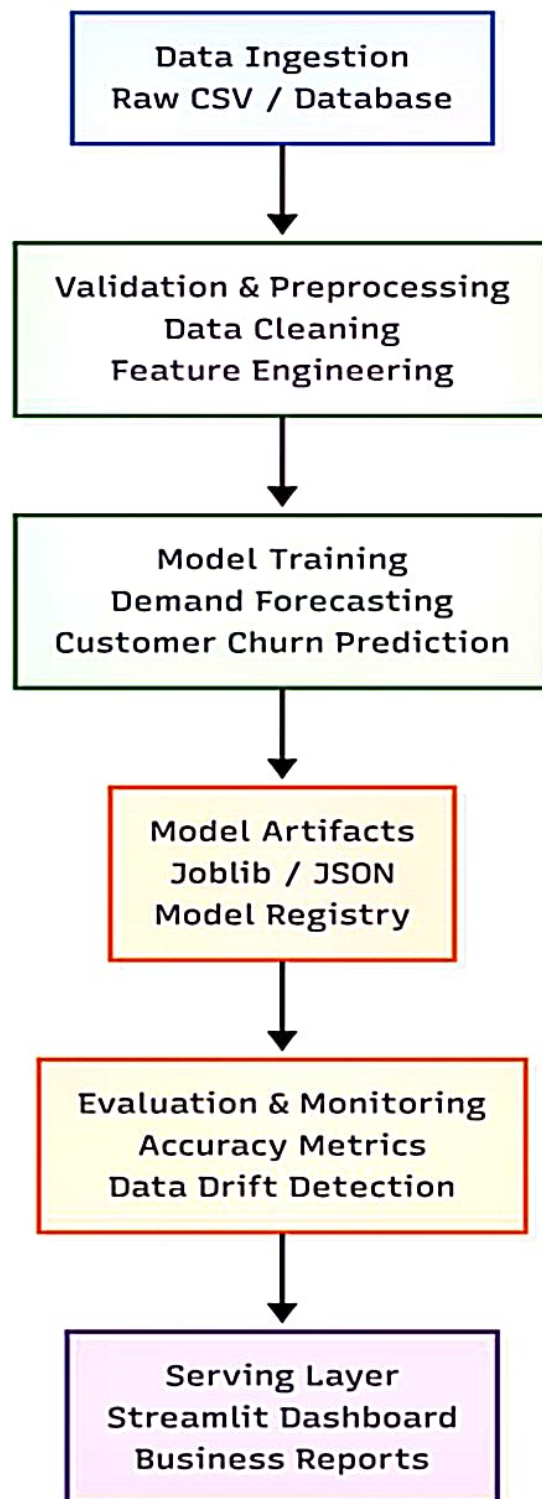


Fig 4.2 MLOps Pipeline Diagram

CHAPTER 5: DETAILED EXECUTION TIMELINE

5.1: Week-by-Week Breakdown:

Week-1: Ingest and validate raw data, perform EDA and feature engineering, produce cleaned datasets and a baseline forecast.

Week-2: Develop and evaluate forecasting and churn models, build inventory optimization logic, and log experiments and metrics.

Week-3: Implement the Streamlit dashboard, integrate model outputs and visualizations, and add export and reporting features.

Week-4: Containerize the app, add CI/CD and Kubernetes manifests, set up monitoring and drift detection, perform load testing and final QA.

5.2: Major Deliverables per Phase:

Data Exploration & Preparation: The project established a robust analytics and forecasting foundation by creating cleaned and validated datasets (`cleaned_transactions.csv`, `daily_sales.csv`, and `customer_features.csv`), conducting exploratory data analysis and quality checks in `week_1.ipynb`, and developing a baseline Prophet model with experiment logging to generate a 30-day forecast and serve as a benchmark for future improvements.

Advance Modelling and Churn Prediction: The project delivered production-ready machine learning assets, including a tuned demand forecasting model (`demand_forecasting_model.json`) for inventory planning, an XGBoost churn prediction model (`xgb_churn_model.joblib`) that generates churn risk scores with SHAP-based explanations, and versioned feature store snapshots with comprehensive evaluation reports containing metrics, cross-validation results, and error analyses to ensure reproducibility and support informed model selection.

Dashboard and Analytics Layer: The project delivered a production-ready Streamlit dashboard (`app.py`, `data_loader.py`, and `ui.py`) that integrates KPIs, visualizations, and machine learning outputs for sales, customer segmentation, forecasting, churn, and inventory management, complemented by interactive reporting features such as Plotly charts, CSV/PDF exports, and scheduled reports. Integration testing and a UX validation checklist were also completed to ensure reliable data refreshes, accurate model outputs, and a seamless user experience prior to deployment.

Deployment, MLOps & Production Polish: The project delivered a production-ready deployment ecosystem, including containerization and orchestration artifacts (Dockerfile, docker-compose.yml, and deployment.yaml) for consistent local and cloud deployments, alongside automated CI/CD workflows and model version management using GitHub Actions and MLflow. Monitoring and reliability were ensured through performance tracking, data drift detection, and load-testing reports using Prometheus, Grafana, and Evidently, while comprehensive documentation and a demo package (README.md and demo video) were provided to support stakeholder onboarding, system adoption, and knowledge transfer.

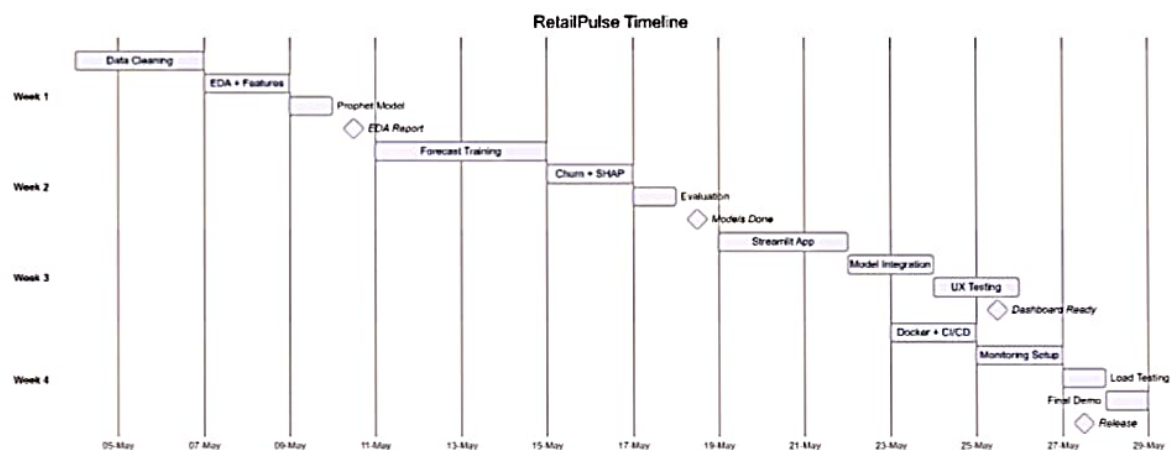


Fig 5.1 Gantt Chart

CHAPTER 6: TECHNICAL HIGHLIGHTS

6.1: Model Performance Metrics:

RetailPulse targets demand forecasting accuracy with a MAPE below 10%, an MAE under 15, an RMSE under 25, and an R^2 above 0.75, while churn prediction aims for an AUC above 0.85, an F1 greater than 0.75, precision above 0.70, recall above 0.75, and overall classification accuracy above 0.80, supporting reliable customer retention, inventory decision-making, and ongoing model validation for operations.

6.2: Feature Engineering Details:

RetailPulse feature engineering creates derived retail signals from raw transaction, customer, and product data, including RFM scores, rolling sales windows, demand seasonality features, inventory velocity, and customer lifetime value. It also encodes product hierarchies, promotion flags, store-level performance ratios, and categorical customer segments. These features are then used for forecasting, churn classification, inventory optimization, and demand planning models, enabling the dashboard to surface actionable retail insights and downstream model accuracy improvements.

6.3: MLOps Practices:

RetailPulse adopts MLOps practices by tracking experiments and model artifacts with MLflow, automating retraining pipelines based on fresh sales and customer data, and monitoring model drift with data-quality checks. Drift detection alerts trigger scheduled retraining when forecast or churn model performance degrades, while versioned models and reproducible data pipelines ensure safe deployment and operational continuity. This supports continuous model improvement, auditability, and reliable analytics for retail operations.

6.4: Challenges Faced & Solutions:

RetailPulse faced noisy, incomplete retail data, slow feature pipeline iterations, and model drift risk from changing demand patterns. These were solved through robust preprocessing, feature validation, automated experiment tracking with MLflow, drift detection checks, and stakeholder review cycles. Containerization, dashboard integration, CI/CD, and reproducible data pipelines reduced deployment friction, while versioned artifacts and retraining workflows ensured stable forecasting and churn scoring across evolving retail scenarios.

CHAPTER 7: DEPLOYMNET AND OPERATIONS

7.1: Platform used:

RetailPulse uses Streamlit Cloud for fast dashboard deployment and prototyping, while AWS is recommended for production-grade scaling, secure hosting, and managed services to support growing retail data, model serving, and monitoring needs.

7.2: CI/CD Pipeline Summary:

RetailPulse CI/CD automates build, test, and deployment of the Streamlit dashboard and model artifacts through GitHub Actions or equivalent pipelines. Commits trigger linting, unit tests, container build, artifact packaging, deployment to staging or production environments, and integration validation with data pipelines. The pipeline also includes model versioning, environment validation, rollback readiness, notification hooks, and security checks, ensuring consistent releases, faster delivery, compliant deployment, and reliable retail analytics operations.

7.3: Monitoring Detection Setup:

RetailPulse monitoring uses automated drift detection and performance tracking across forecast and churn models, with tools like Evidently or custom monitoring dashboards. It collects data quality metrics, input distribution changes, and prediction deviations, then alerts when models drift beyond thresholds. Combined with logging and periodic validation reports, this setup enables timely retraining, preserves model accuracy, and keeps retail analytics aligned with changing customer behavior and demand patterns.

CHAPTER 8: VISUALS

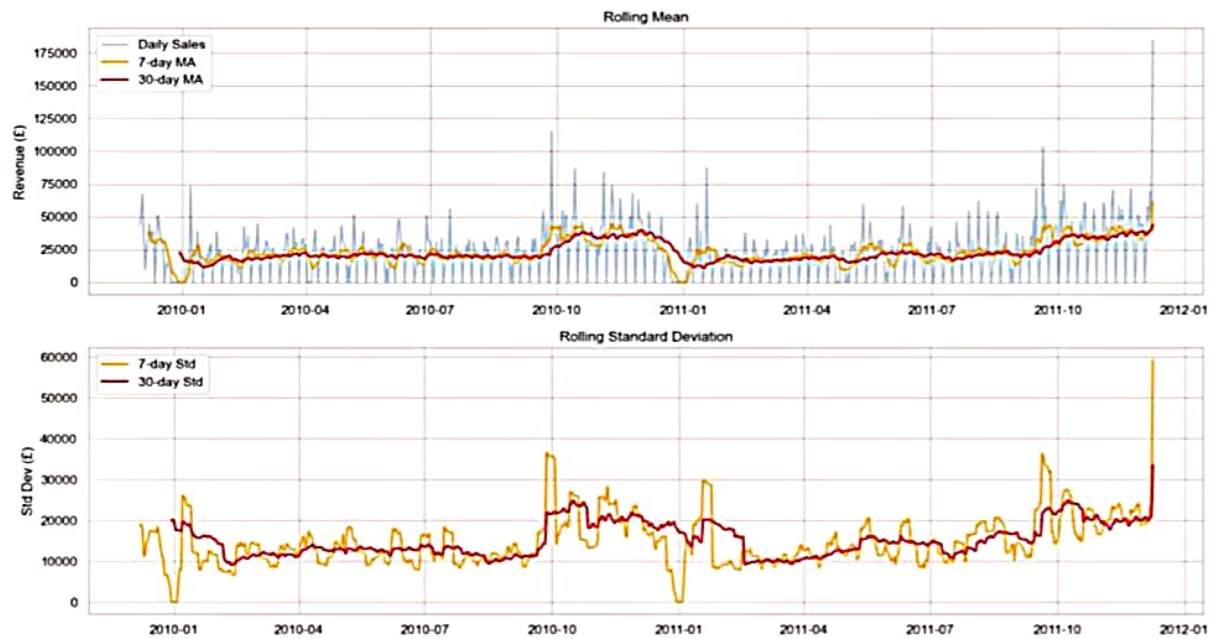


Fig 8.1 Rolling Mean and STD

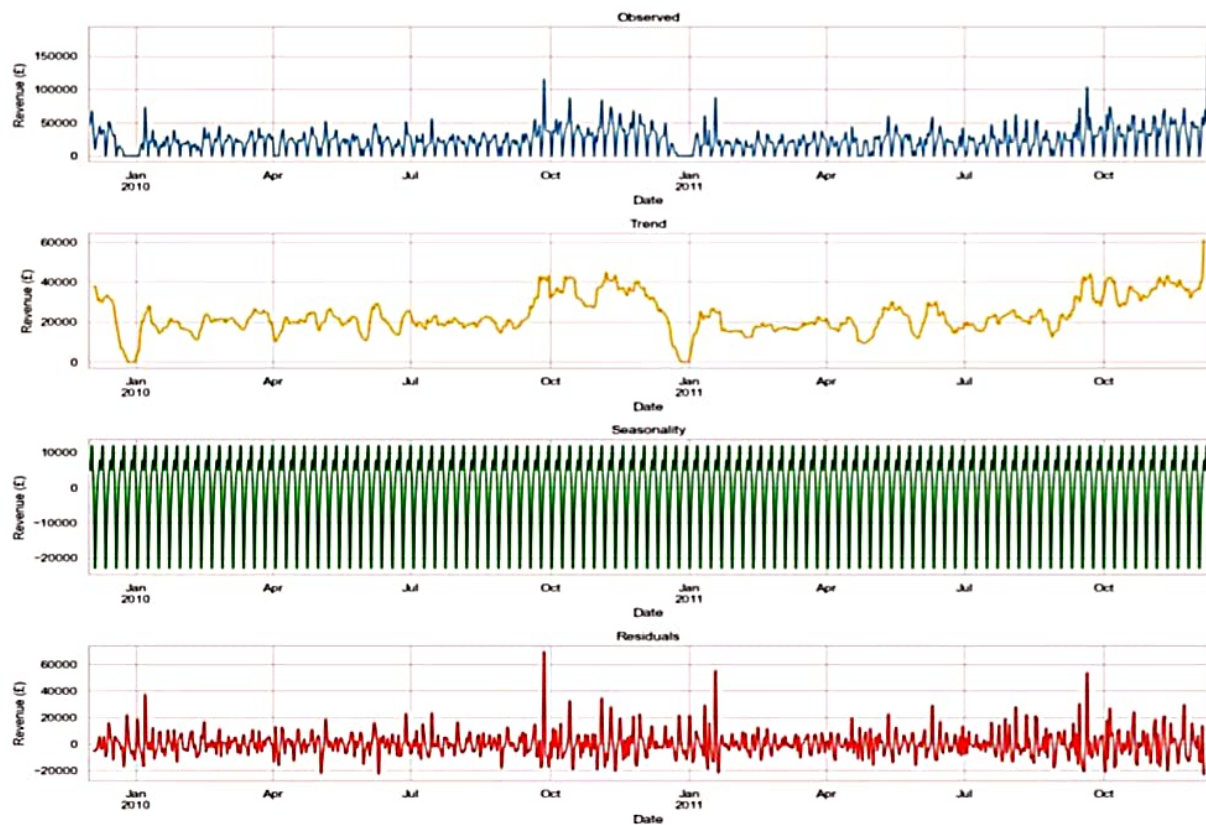


Fig 8.2 Seasonal Decomposition

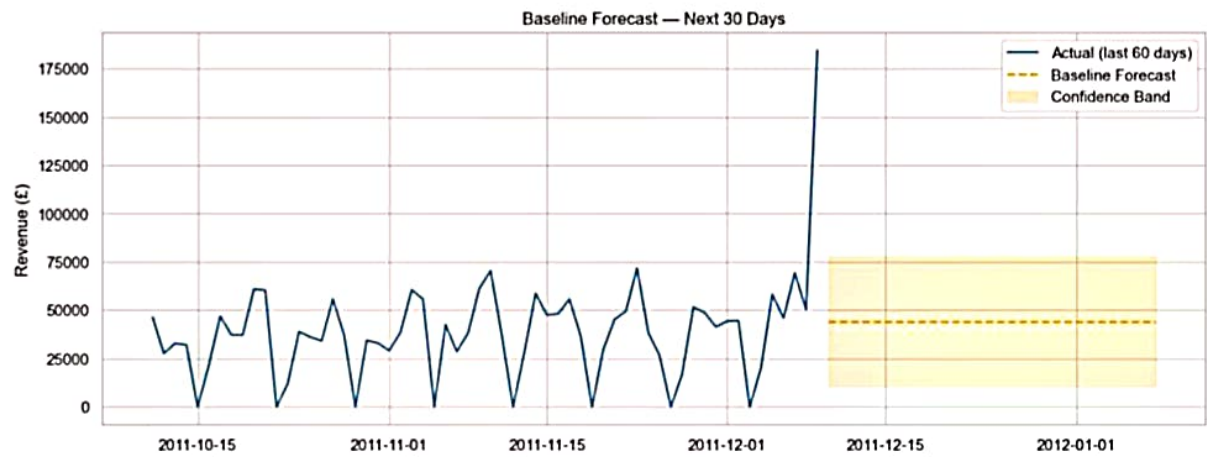


Fig 8.3 Baseline Forecast Plot

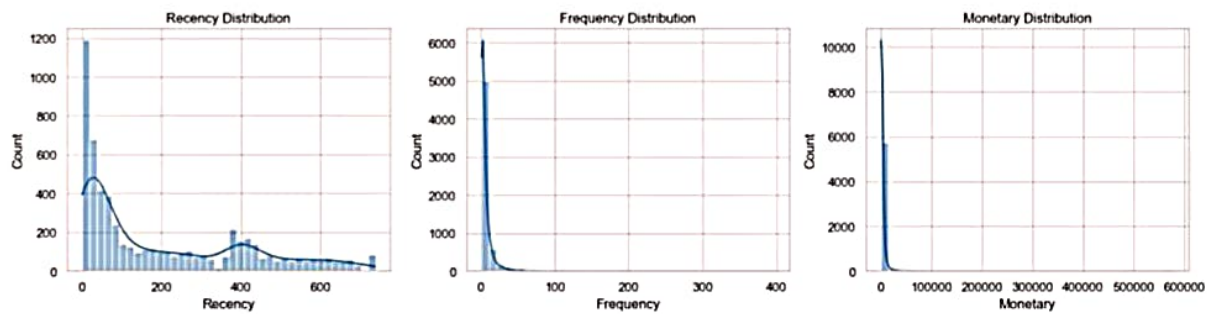


Fig 8.4 RFM Plot for Raw Data

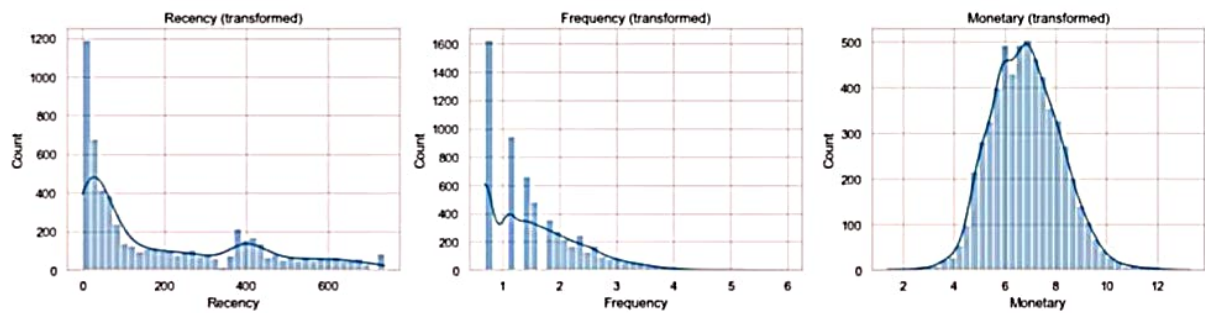


Fig 8.5 RFM Plot for Transformed Data

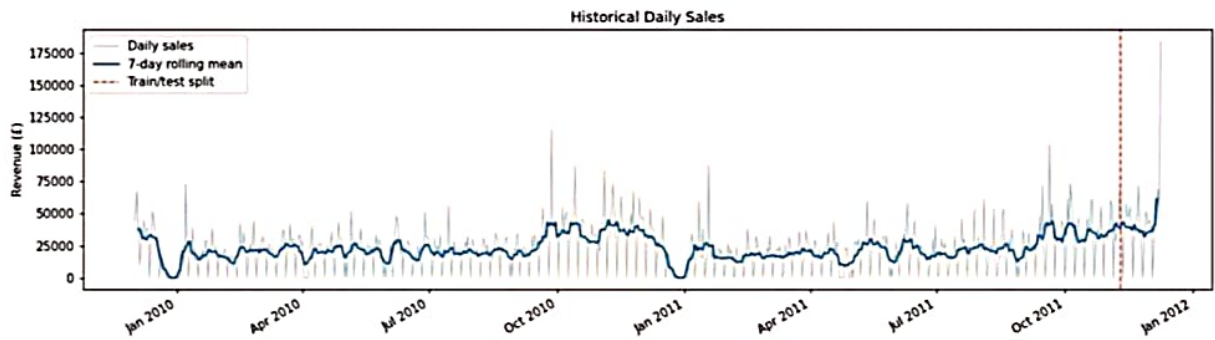


Fig 8.5 Historical Trends

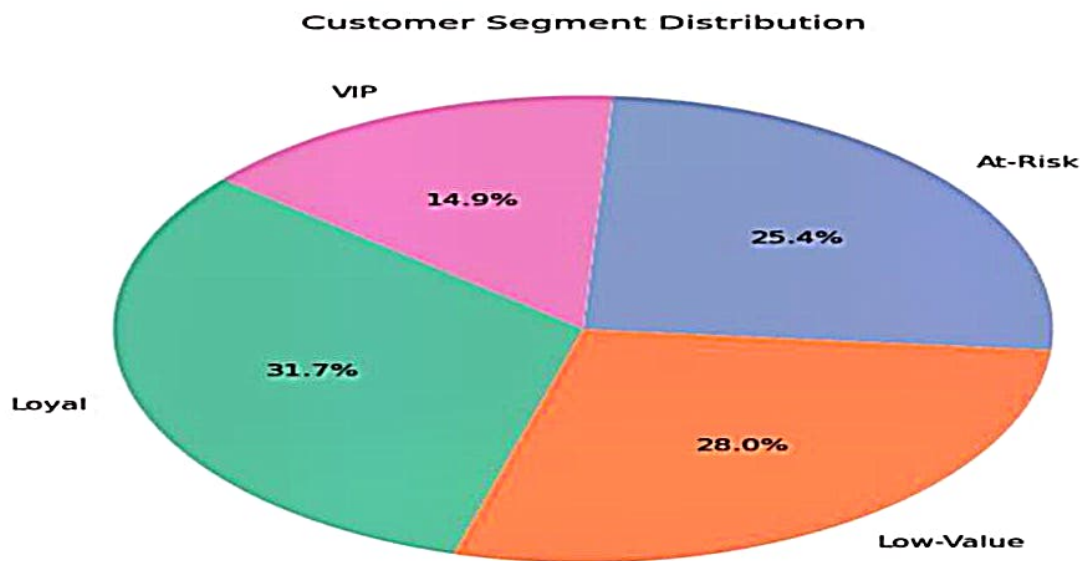


Fig 8.6 Customer Clusters

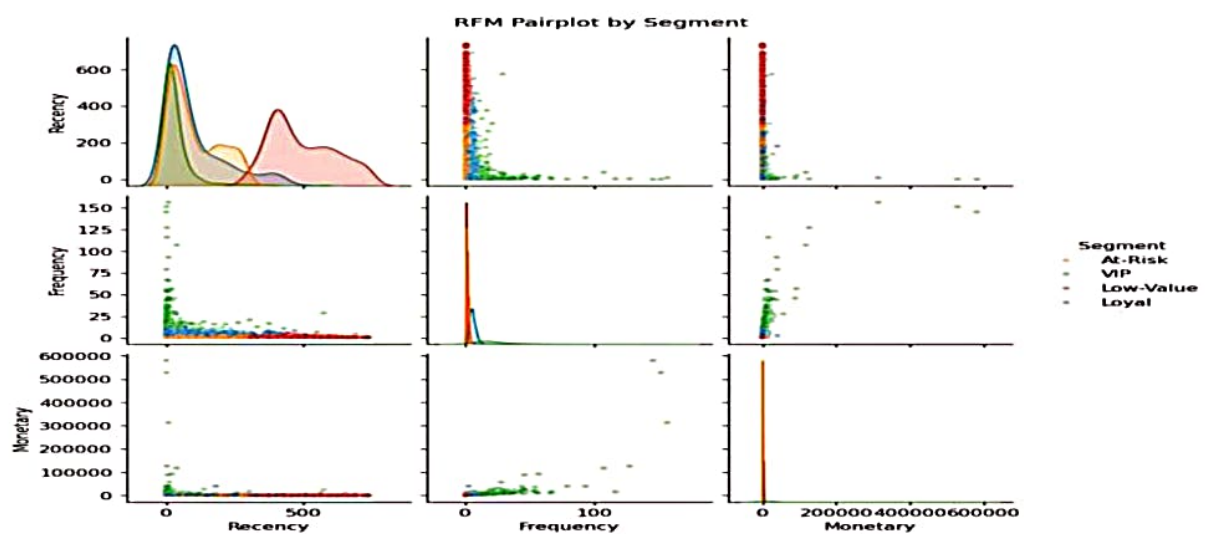


Fig 8.7 RFM Pair Plot

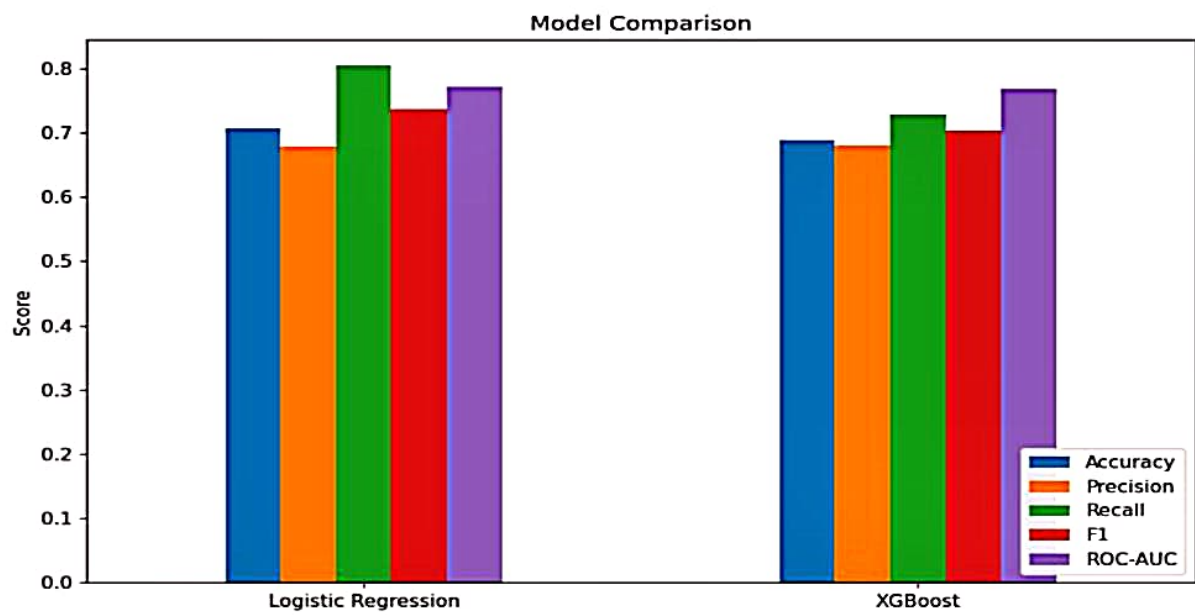
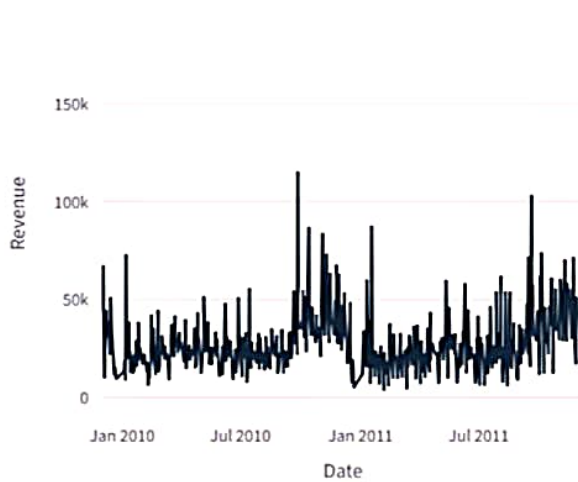


Fig 8.8 Model Comparison

Daily Sales Trend

Daily Revenue



Revenue by Country (Top 10)

Top 10 Countries by Revenue

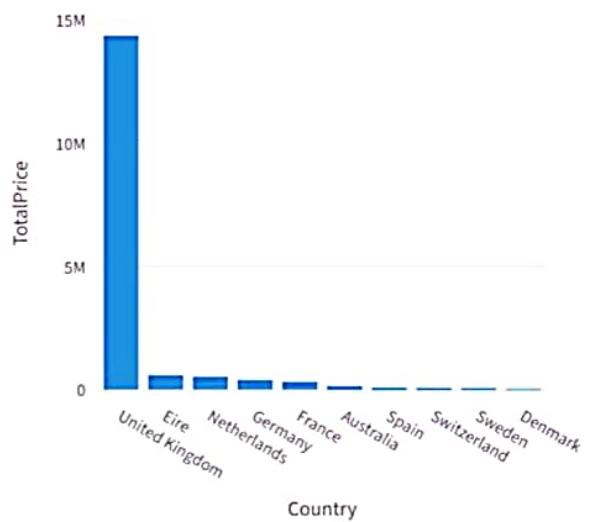


Fig 8.9 Daily Sales Trend and Revenue by Country

Monthly Revenue

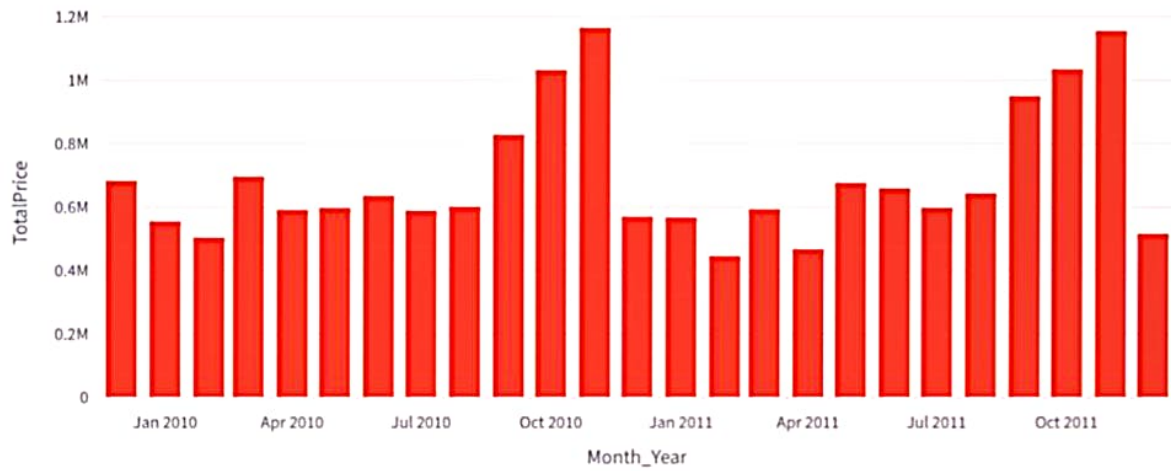


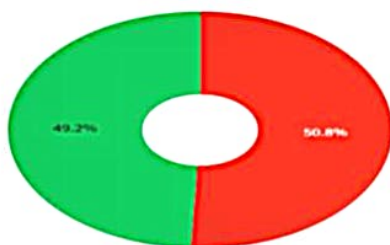
Fig 8.10 Monthly Sales Trend

Forecast Trend with Confidence Intervals



Fig 8.11 Forecast Trend

Customer Status Distribution



Avg Basket Size: Active vs Churned

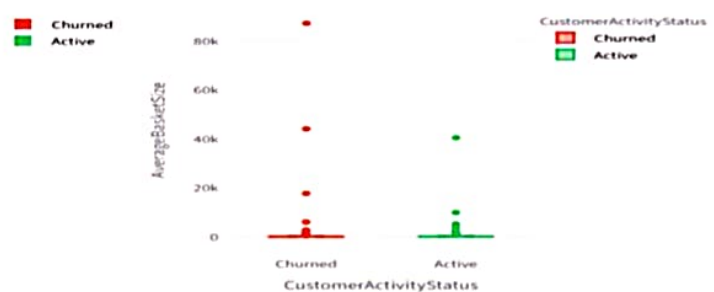


Fig 8.11 Churn Analysis

Product Velocity and Revenue

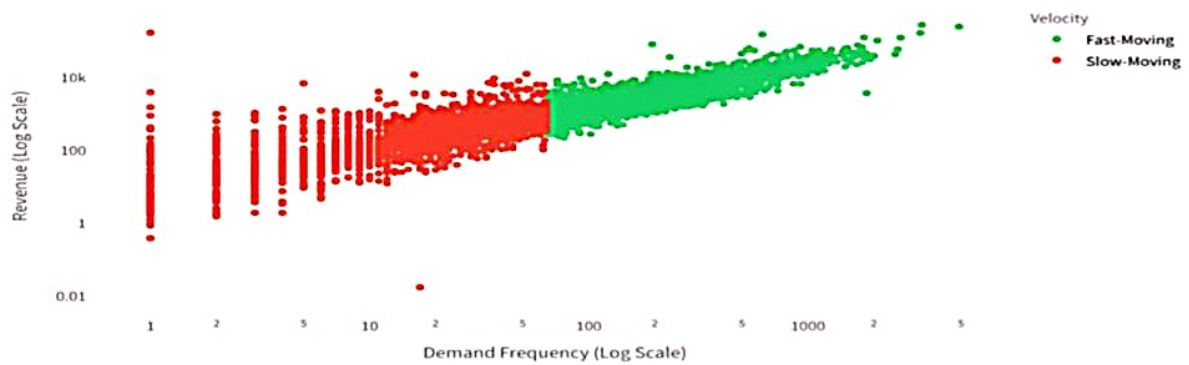


Fig 8.12 Revenue Demand Frequency

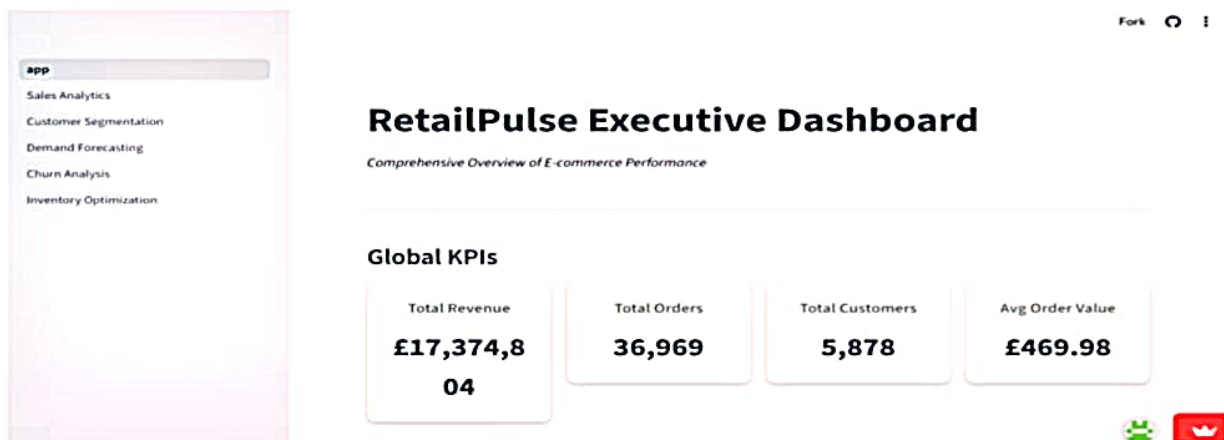


Fig 8.13 Dashboard Page 1

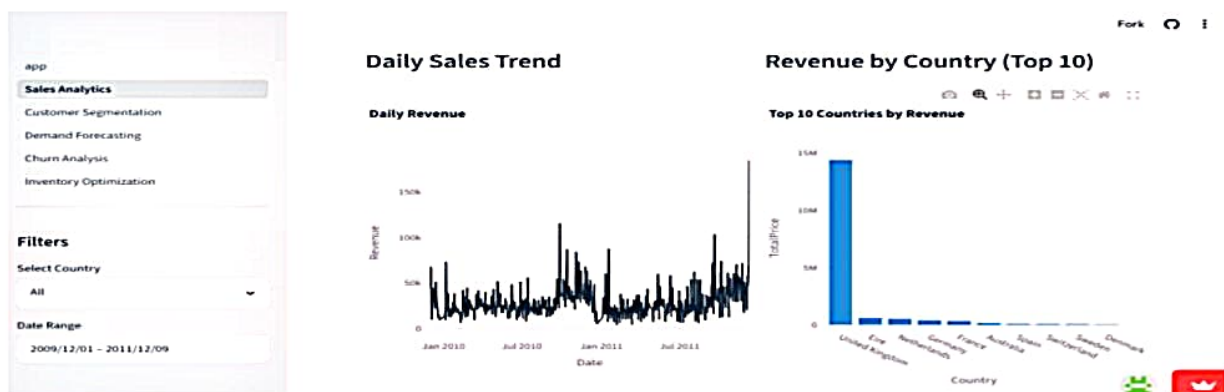
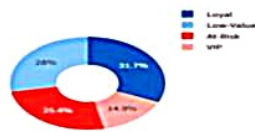


Fig 8.14 Dashboard Page 2



Segment Distribution

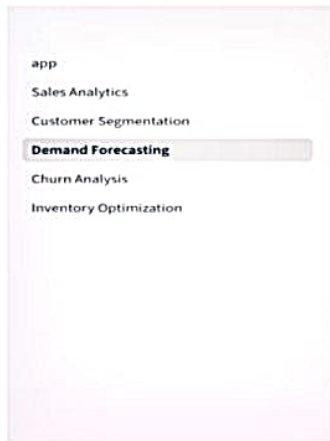
Customers by Segment



Customer Directory

CustomerID	Segment	Recency	Frequency	Monetary
12346	VIP	325	12	77356.46
12347	Loyal	1	8	4921.53
12348	Loyal	74	5	2019.4
12349	Loyal	18	4	4428.69
12350	Low-Value	809	1	334.4
12351	Low-Value	374	1	300.93
12352	Loyal	35	10	2849.84
12353	At Risk	203	2	406.76
12354	At Risk	231	1	1079.4
12355	At Risk	213	2	947.61

Fig 8.15 Dashboard Page 3



Demand Forecasting

30-Day Sales Predictions

30-Day Sales Forecast

Predicted 30-Day Revenue

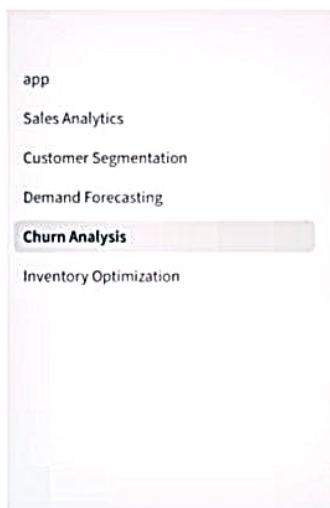
£417,457

Average Daily Predicted

£13,915

Forecast Trend with Confidence Intervals

Fig 8.16 Dashboard Page 4



Churn Analysis

Customer Attrition Metrics

Churn Statistics

Overall Churn Rate

50.8%

Total Churned Customers

2,985

Fig 8.16 Dashboard Page 4

CHAPTER 9: PERSONAL REFLECTIONS

Working on RetailPulse has been a valuable exercise in bridging retail analytics with practical deployment and MLOps. I learned to turn raw transaction and customer data into meaningful features, build forecasting and churn models, and deliver insights through a Streamlit dashboard. Managing versioned artifacts, drift monitoring, and deployment workflows reinforced the importance of reproducibility and operational reliability. Collaborating with stakeholder requirements and adapting models for real retail scenarios also improved my ability to create data-driven solutions that are actionable, scalable, and user-focused.